

LAPORAN TUGAS TAMBAHAN

Deploy Website pada Cloud/VPS dan Konfigurasi Web Server

Studi Kasus: Website Abadi Cell & Elektrik

Mata Kuliah	Jaringan Komputer
Nama	Arif Mufti Tharsa
NIM	105224041
Program Studi	S1 Ilmu Komputer
Universitas	Universitas Pertamina
Semester	4 (Empat)
Tanggal Pengumpulan	16 Juli 2026
URL Website	https://abadicellelektrik.my.id
Repository GitHub	github.com/arifmuftitharsa/abadi-cell-website

1. Identitas & Deskripsi Website

Website ini dibuat untuk Abadi Cell & Elektrik, toko elektronik, aksesoris, dan pulsa milik keluarga penulis di Sukabumi, Jawa Barat. Website menampilkan empat layanan toko: Pulsa & Isi Saldo, Aksesoris HP, Elektronik Rumah Tangga, dan Jasa Servis. Untuk kategori Jasa Servis, toko berperan sebagai penghubung ke teknisi servis di luar toko, bukan mengerjakan servis sendiri. Website juga menampilkan lokasi toko, jam operasional, dan kontak WhatsApp.

Selain untuk tugas ini, website menjadi bagian awal dari rencana digitalisasi toko yang lebih besar. Rencana selanjutnya adalah membuat sistem pencatatan transaksi digital dan ringkasan penjualan sederhana.

2. Infrastruktur yang Digunakan

Website di-deploy menggunakan Railway, sebuah platform cloud hosting (Platform-as-a-Service/PaaS). Arsitektur akhir terdiri dari dua service yang saling terhubung lewat jaringan privat, bukan satu VPS mentah yang dikonfigurasi penuh dari command line. Perbedaan ini penulis sampaikan secara terbuka karena berpengaruh pada level penilaian di rubrik tugas.

Komponen	Detail
Platform Hosting	Railway (Platform-as-a-Service)
Arsitektur	2 service dalam 1 project: app (privat) dan proxy (publik)
Domain	abadicellelektrik.my.id, dibeli lewat Hostinger
Web server service app	Nginx 1.27 Alpine, menyajikan file HTML/CSS/JS statis
Web server service proxy	Caddy 2 Alpine, reverse proxy ke service app
Containerization	Docker, dengan 2 Dockerfile terpisah
Jaringan antar-service	Railway Private Networking, DNS internal *.railway.internal
Sertifikat HTTPS	Dikelola otomatis oleh Railway
CI/CD	GitHub Actions, validasi build Docker dan Caddyfile tiap push
Monitoring	UptimeRobot, mengecek endpoint /health tiap 5 menit
DNS Domain	Dikelola di Hostinger (TXT, CNAME, ALIAS)

Karena memakai Railway, HTTPS diurus otomatis oleh platform, tidak dipasang manual dengan Certbot seperti pada VPS mentah. Yang penulis kerjakan dan konfigurasi sendiri adalah containerization dengan Docker, pemisahan arsitektur jadi dua service, konfigurasi Caddy sebagai reverse proxy, jaringan privat antar-service, CI/CD, DNS domain, dan monitoring.

3. Tahapan Deployment

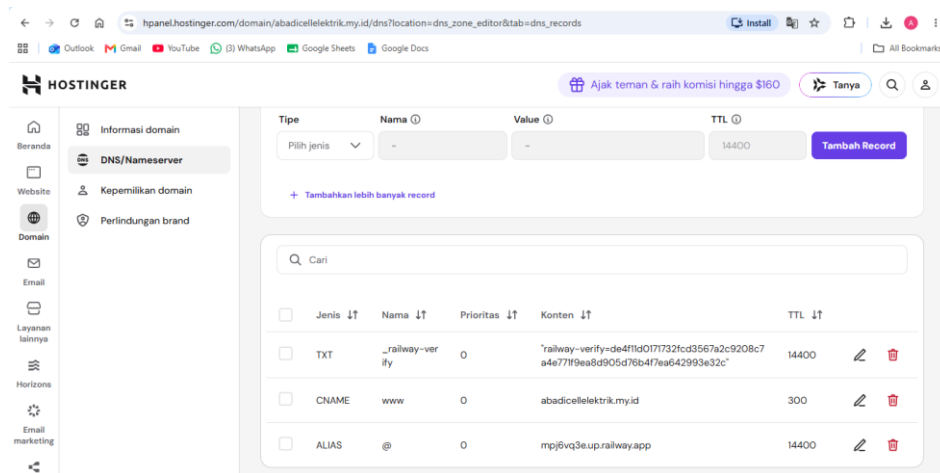
3.1 Containerization Awal dengan Docker

Website statis dibungkus jadi Docker image dengan base image nginx:1.27-alpine. Image Alpine dipilih karena ukurannya kecil, sekitar 20MB, dan cukup ringan untuk kebutuhan hosting sederhana seperti ini.

```
FROM nginx:1.27-alpine
RUN rm /etc/nginx/conf.d/default.conf
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
COPY app/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

3.2 Deploy ke Railway dan Konfigurasi Domain

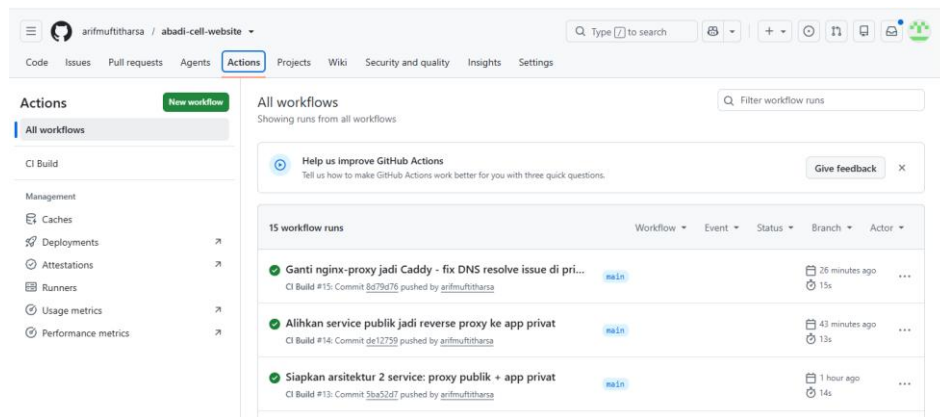
Repository GitHub dihubungkan ke Railway, sehingga setiap push ke branch main otomatis memicu build dan deploy. Domain .my.id dari Hostinger dihubungkan ke Railway lewat tiga DNS record: TXT untuk verifikasi kepemilikan domain, CNAME untuk subdomain www, dan ALIAS pada root domain yang mengarah ke endpoint Railway.



Gambar 3.1. Konfigurasi DNS domain di Hostinger

3.3 CI/CD dengan GitHub Actions

Workflow GitHub Actions dibuat untuk menjalankan validasi build Docker image dan validasi Caddyfile setiap ada push ke branch main, sebelum Railway melakukan auto-deploy. Tujuannya supaya kesalahan konfigurasi ketahuan lebih dulu di tahap CI, bukan langsung gagal saat sudah live.



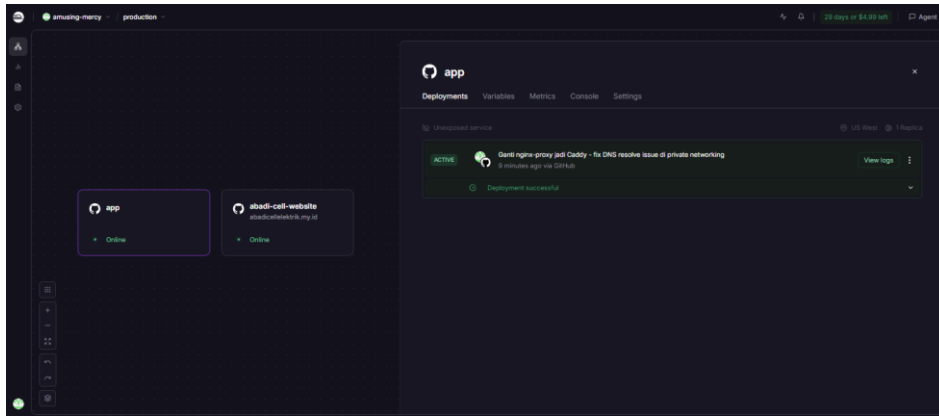
Gambar 3.2. Riwayat GitHub Actions, seluruh run berhasil

3.4 Migrasi ke Arsitektur Reverse Proxy Dua Service

Arsitektur diubah dari satu container menjadi dua service dalam satu project Railway. Tujuannya memisahkan service yang menghadap publik dari service yang menyimpan file aplikasi, sekaligus mengejar poin bonus Reverse Proxy dan Multi-Service Deployment.

- Service app (privat): Nginx yang menyajikan file statis. Tidak punya domain publik, hanya bisa diakses lewat jaringan privat Railway di app.railway.internal.
- Service proxy (publik): menghadap internet, memegang domain abadicellektrik.my.id, dan meneruskan semua request ke service app lewat private networking.

Kedua service terhubung lewat fitur Private Networking Railway, yaitu jaringan mesh terenkripsi dengan resolusi DNS internal berdasarkan nama service.



Gambar 3.3. Service app dan service publik berjalan online dalam satu project Railway yang sama

3.5 Konfigurasi Reverse Proxy dengan Caddy

Percobaan pertama reverse proxy memakai Nginx pada service publik sempat gagal karena masalah resolusi DNS internal. Proses debugging-nya dijelaskan lebih rinci di bagian Hambatan dan Solusi. Setelah dianalisis, service publik akhirnya memakai Caddy 2, karena Caddy melakukan pencarian DNS ulang di setiap request tanpa perlu konfigurasi resolver manual seperti di Nginx.

```

:{$PORT}
auto_https off
encode gzip

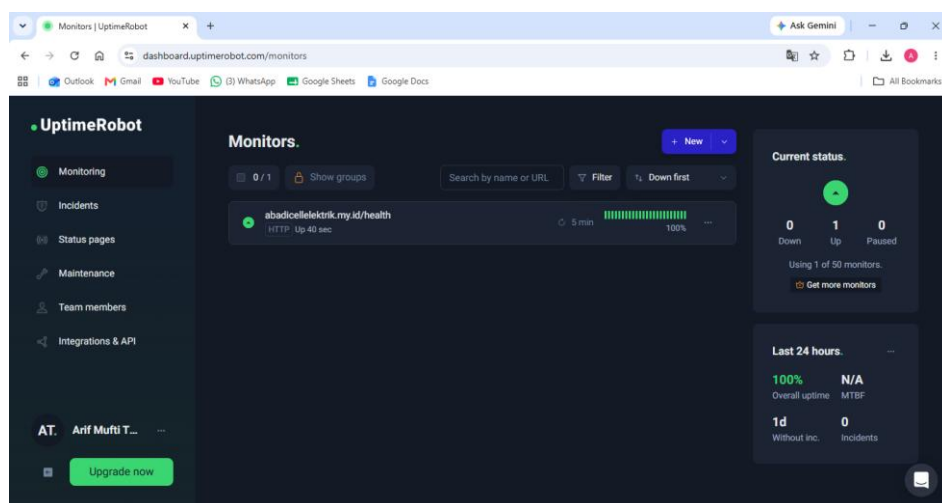
handle /health {
    respond "ok" 200
}

handle {
    reverse_proxy app.railway.internal:8080
    header X-Frame-Options SAMEORIGIN
    header X-Content-Type-Options nosniff
    header Referrer-Policy strict-origin-when-cross-origin
}

```

3.6 Setup Monitoring dengan UptimeRobot

Sebagai lapisan monitoring tambahan, dibuat monitor HTTP di UptimeRobot yang mengecek endpoint <https://abadicellektrik.my.id/health> tiap 5 menit. Dengan begitu status website bisa dipantau dari luar Railway sendiri.

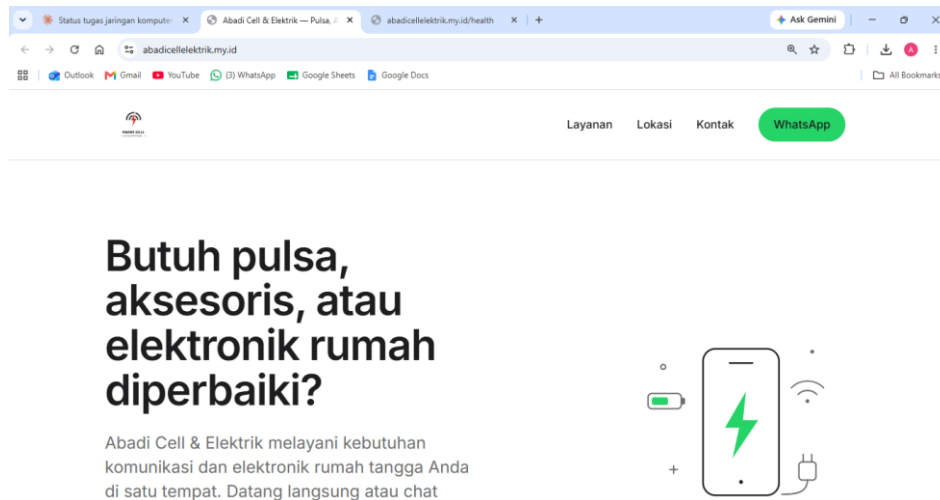


Gambar 3.4. Dashboard UptimeRobot, status Up dengan uptime 100%

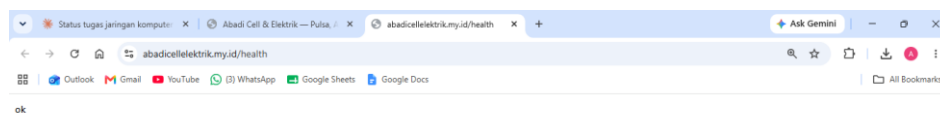
4. Bukti Running

Website dapat diakses secara publik pada saat laporan ini ditulis, melalui URL berikut:

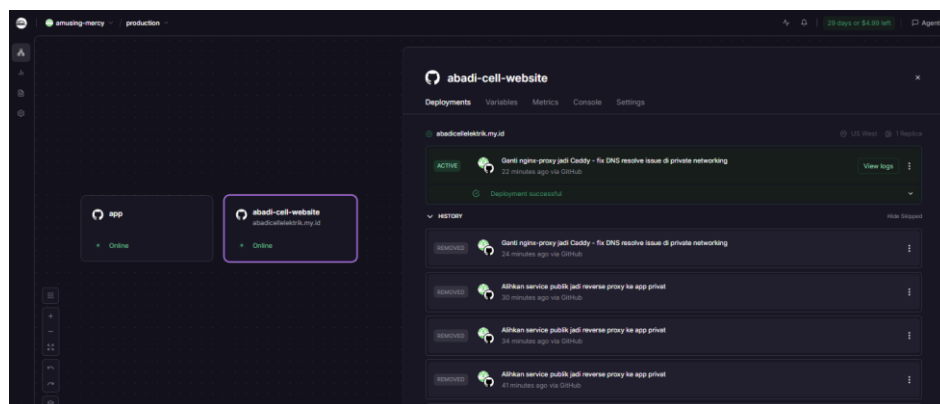
<https://abadicellelektrik.my.id>



Gambar 4.1. Halaman utama website Abadi Cell & Elektrik



Gambar 4.2. Endpoint /health merespons "ok"



Gambar 4.3. Dashboard Railway, service publik berstatus Active

5. Hambatan dan Solusi

Bagian ini menjelaskan hambatan teknis yang dialami selama proses deployment, terutama pada tahap migrasi ke arsitektur reverse proxy dua service.

5.1 502 Bad Gateway saat Migrasi Reverse Proxy

Setelah service publik diubah jadi reverse proxy dengan Nginx, website menampilkan error 502 Bad Gateway. Log deployment menunjukkan pesan berikut.

```
app.railway.internal could not be resolved (3: Host not found)
```

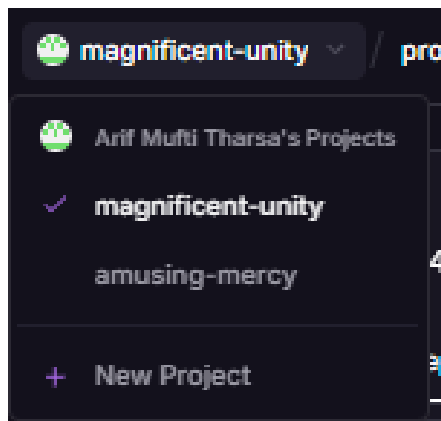


Gambar 5.1. Error 502 Bad Gateway pada percobaan pertama

Penulis mencari referensi di forum komunitas Railway. Beberapa pengguna lain melaporkan masalah serupa: Nginx berbasis Alpine kadang gagal melakukan pencarian ulang DNS internal Railway ketika IP privat sebuah service berubah. Solusi yang dicoba adalah mengganti web server proxy dari Nginx ke Caddy, karena Caddy melakukan pencarian DNS ulang di setiap request secara bawaan. Namun setelah diganti ke Caddy, error yang sama masih muncul di log.

```
dial tcp: lookup app.railway.internal on [fd12::10]:53: no such host
```

Munculnya error yang sama persis di dua web server berbeda menjadi petunjuk bahwa masalahnya bukan di pilihan web server. Setelah dicek ulang, ternyata nama project di dashboard Railway berbeda antara service app dan service publik. Service app tanpa sengaja dibuat sebagai project baru yang terpisah, karena penulis salah klik tombol "New Project", bukan tombol tambah service di dalam project yang sedang terbuka.



Gambar 5.2. Service app ternyata berada di project Railway yang berbeda dari project utama

Private networking Railway hanya berfungsi antar-service yang ada di project dan environment yang sama. Dua service di project yang berbeda tidak akan pernah bisa saling menemukan lewat DNS internal, apa pun web server yang dipakai. Setelah service app dihapus dan dibuat ulang di project yang benar, private networking langsung berfungsi dan website bisa diakses normal.

5.2 Menjaga Website Tetap Online Selama Percobaan

Karena website harus tetap bisa diakses saat dinilai, setiap perubahan arsitektur dibatasi maksimal dua kali percobaan. Kalau dalam dua kali percobaan itu masih gagal, konfigurasi akan dikembalikan ke versi Dockerfile lama yang sudah terbukti stabil. Aturan ini dibuat supaya tetap bisa mengejar bonus poin tanpa mengorbankan risiko website mati saat dinilai.

5.3 Verifikasi Akademik GitHub Student Developer Pack

Di luar proses deployment, pengajuan GitHub Student Developer Pack sempat ditolak dua kali. Penolakan pertama karena dokumen yang diunggah terdeteksi berasal dari galeri foto, bukan hasil kamera langsung. Penolakan kedua karena dokumen terdeteksi sebagai surat penerimaan kuliah, bukan bukti keaktifan studi saat ini. Hal ini tidak berpengaruh ke proses deployment, tapi dicatat sebagai bagian dari proses mengurus dokumen teknis dan administratif secara bersamaan.

6. Refleksi Pembelajaran

Tugas ini membantu penulis memahami konsep Application Layer yang sebelumnya hanya dipelajari lewat teori. Proses debugging error 502 menunjukkan langsung bagaimana satu permintaan HTTP dari klien harus melewati beberapa lapisan: DNS publik, reverse proxy, DNS privat, baru sampai ke server aplikasi. Kalau salah satu lapisan gagal, seluruh permintaan ikut gagal.

Konsep DNS jadi lebih jelas setelah menangani dua jenis resolusi DNS dalam satu tugas ini. Pertama, DNS publik untuk mengarahkan domain `abadicellelektrik.my.id` ke Railway lewat TXT, CNAME, dan ALIAS record di Hostinger. Kedua, DNS privat untuk resolusi nama service seperti `app.railway.internal`, yang hanya berlaku di dalam satu project Railway yang sama. Kesalahan membuat service di project yang berbeda mengajarkan penulis pentingnya memahami batas cakupan suatu jaringan, sebelum berasumsi dua sistem pasti bisa saling terhubung.

Reverse proxy jadi lebih konkret dibanding sekadar istilah di buku. Penulis memahami polanya sebagai pemisahan antara service yang menghadap publik dan service yang menyimpan aplikasi, sehingga aplikasi tidak pernah terekspos langsung ke internet. Pengalaman berpindah dari Nginx ke Caddy juga mengajarkan bahwa dua software dengan fungsi serupa bisa punya perilaku internal yang berbeda, terutama soal caching DNS.

Docker dipahami bukan cuma sebagai cara membungkus aplikasi, tapi sebagai unit isolasi yang membuat dua service dengan tanggung jawab berbeda bisa berjalan sendiri-sendiri, sambil tetap bisa berkomunikasi lewat jaringan yang diatur dengan jelas. Penerapan CI/CD dengan GitHub Actions juga memberi pelajaran soal pentingnya validasi otomatis sebelum kode benar-benar dijalankan di production.

Penulis menyadari bahwa arsitektur akhir memakai Railway sebagai Platform-as-a-Service, bukan VPS yang dikelola sendiri sepenuhnya dari command line. Pilihan ini diambil setelah mengecek opsi VPS gratis, yang semuanya mensyaratkan kartu kredit untuk verifikasi, dan setelah VPS lama yang sempat dicoba ternyata sudah tidak aktif. Pertimbangan utamanya adalah keterbatasan waktu dan biaya. Meski begitu, containerization, reverse proxy dua service, private networking, CI/CD, dan monitoring tetap dikerjakan sebagai usaha memahami konsep jaringan komputer secara langsung, bukan sekadar deploy satu klik tanpa mengerti apa yang terjadi di baliknya.